计算机网络实验 1
利用流式套接字编写聊天程序

鲁昕宁 2414015

2026 年 10 月 20 日

1 实验目的

用 C++ 和 Winsock 基本套接字实现一个多人聊天程序，服务端与客户端都是 WinUI 3 窗口。实验要完成四件事：

1. 为 TCP 字节流设计带长度的应用层帧，使一条消息不会被粘在一起或从中间拆开；
2. 用多线程同时服务多个客户，界面线程只负责显示；
3. 支持英文和中文；
4. 用每条帧上的序号观察连接存活时有没有丢消息。

2 实验环境

程序使用 C++17。套接字只调用 Winsock2 的 `socket`、`bind`、`listen`、`accept`、`connect`、`send`、`recv` 和 `closesocket`。每个客户连接由一个 `std::thread` 接收，界面更新通过 WinUI 的 `DispatcherQueue.TryEnqueue` 回到 UI 线程。

编译环境为 Visual Studio 2022 Professional，平台工具集 v143，Windows SDK `10.0.22621.0`，Windows App SDK 1.6。工程是自包含运行库的桌面程序。可执行文件放在 `bin` 目录下，分别是服务端和客户端。

3 聊天协议

TCP 交给应用的是字节流。一次 `recv` 可能只得到半条消息，也可能一次得到多条。协议因此使用固定 10 字节头加 UTF-8 负载。多字节整数使用网络字节序。负载最长 1024 字节。

表 1: 帧头布局

偏移	长度	含义
0	2	魔数 `0x43 0x48`（`CH`）
2	1	版本，当前为 1
3	1	消息类型
4	4	序号 `uint32`，每条连接的每个方向从 1 递增
8	2	负载长度 `uint16`
10	n	UTF-8 负载

发送时循环调用 `send`，直到 10 字节头和全部负载都写完。接收时先读满 10 字节，校验魔数、版本和长度，再按长度读满负载。魔数、版本或长度非法时，服务端关闭该连接。

昵称是 1 到 16 个 Unicode 字符，不允许控制字符。重名会被拒绝。默认端口是 9000。客户点“断开”或关闭窗口时先发 QUIT，再关闭套接字并等待接收线程结束。服务端点“停止”时停止 `accept`，关闭已有连接。`recv` 返回 0 或错误时，服务端从客户表删除该连接，并向其他人广播离开通知。

表 2: 消息类型

值	名称	作用
1	HELLO	客户登记昵称
2	WELCOME	服务端确认登记
3	CHAT	聊天文本。转发时负载为“昵称: 正文”
4	NOTICE	加入、离开或在线列表
5	LIST	客户请求在线列表
6	QUIT	客户主动退出
7	ERR	昵称、编码或长度不合法

4 程序结构

4.1 源码目录

源码按“协议和套接字核心”与“窗口”分开。窗口工程只负责输入和显示，套接字调用集中在 shared。这样服务端窗口、客户端窗口和环回测试可以共用同一套组帧与收发代码。

```
lab1/
  Chat.sln                解决方案：服务端、客户端、环回测试
  build.bat               调用 MSBuild 生成 Release x64
  shared/
    protocol.h            帧布局、消息类型、默认端口
    netutil.h/.cpp        组帧、拆帧、UTF-8 与 UTF-16 转换
    server_core.h/.cpp    监听、客户表、转发
    client_core.h/.cpp    连接、发送、接收、序号统计
  server/                 WinUI 服务端窗口
    App.xaml.cpp          启动应用
    MainWindow.xaml.cpp   端口、在线用户、日志
  client/                 WinUI 客户端窗口
    App.xaml.cpp          启动应用
    MainWindow.xaml.cpp   连接、消息列表、连续发送
  tests/
    loopback.cpp          不带界面的 100 条环回测试
  bin/                    编译得到的可执行文件和运行库
```

server 与 client 各自是一个 WinUI 工程。MainWindow.idl 声明窗口运行时类，界面在 MainWindow.xaml.cpp 里用代码搭出来。pch.h 先包含 Winsock2，再包含 WinUI 头文件，避免 windows.h 抢先声明套接字类型。tests/loopback.cpp 直接调用 ChatServer 和 ChatClient，用来在没有窗口的情况下核对中文消息和序号。bin 是输出目录，其中的动态库是自包含运行库，和源码分开。

4.2 组帧与拆帧

send 和 recv 都不保证一次调用搬完调用者给出的全部字节。send_all 和 recv_exact 用循环把短发送、短接收补齐，失败或对端关闭时返回假。

```cpp
bool send_all(SOCKET socket, const char* data, int size) {
    int sent_total = 0;
    while (sent_total < size) {
        const int sent = send(socket, data + sent_total, size - sent_total, 0);
        if (sent == SOCKET_ERROR || sent == 0) {
            return false;
        }
        sent_total += sent;
    }
    return true;
}

bool recv_exact(SOCKET socket, char* data, int size) {
    int got_total = 0;
    while (got_total < size) {
        const int got = recv(socket, data + got_total, size - got_total, 0);
        if (got == SOCKET_ERROR || got == 0) {
            return false;
        }
        got_total += got;
    }
    return true;
}
```

`send_frame` 按协议填写 10 字节头。序号和长度写成网络序，然后先后写出头部和负载。`recv_frame` 先读满头部，再按头部中的长度读负载。魔数、版本、类型或长度不合法时返回协议错误，服务端随即关闭这条连接。粘在同一次 `recv` 里的多条消息，会在后面的循环中被拆开。只到达一半的消息会留在循环里，直到剩余字节到齐。

```cpp
RecvStatus recv_frame(SOCKET socket, Frame& frame) {
    uint8_t header[kHeaderSize]{};
    if (!recv_exact(socket, reinterpret_cast<char*>(header),
    ↪  static_cast<int>(kHeaderSize))) {
        return RecvStatus::Closed;
    }
    if (header[0] != kMagic0 || header[1] != kMagic1 || header[2] != kVersion) {
        return RecvStatus::ProtocolError;
    }
    const auto type = static_cast<MsgType>(header[3]);
    switch (type) {
    case MsgType::Hello:
    case MsgType::Welcome:
    case MsgType::Chat:
    case MsgType::Notice:
    case MsgType::List:
    case MsgType::Quit:
    case MsgType::Err:
        break;
    default:
```

```
169         return RecvStatus::ProtocolError;
170     }
171     const uint16_t length = read_u16(header + 8);
172     if (length > kMaxPayload) {
173         return RecvStatus::ProtocolError;
174     }
175     std::string payload(length, '\0');
176     if (length > 0 &&
177         !recv_exact(socket, payload.data(), static_cast<int>(length))) {
178         return RecvStatus::Closed;
179     }
180     frame.type = type;
181     frame.seq = read_u32(header + 4);
182     frame.payload = std::move(payload);
183     return RecvStatus::Ok;
184 }
```

4.3 服务端的客户线程与转发

`accept_loop` 每接受一个连接，就把客户放进客户表，并单独启动接收线程。客户表由互斥锁 `clients_mu_` 保护。每个客户另有发送锁 `send_mu`。`send_to` 持有这把锁之后才写套接字，因此两个线程不会同时写同一个连接。

CHAT 先检查 UTF-8，再在正文前加上发送者昵称，然后 `broadcast`。广播时先在锁内复制客户表，锁外再逐个发送，避免发送阻塞期间一直占着客户表。参数 `except` 是发送者本人，所以消息不会回显。连续发送只在第 1 条、最后一条和每 20 条写日志。

```
203         if (frame.type == MsgType::Chat) {
204             const Utf8Check check = inspect_utf8(frame.payload);
205             if (!check.valid || check.has_control || frame.payload.empty()) {
206                 send_to(client, MsgType::Err,
                    ↪  wide_to_utf8(L"消息需要是不含控制字符的 UTF-8 文本"));
207                 continue;
208             }
209             const std::string outgoing = client->name + ": " + frame.payload;
210             if (outgoing.size() > kMaxPayload) {
211                 send_to(client, MsgType::Err, wide_to_utf8(L" 消息太长"));
212                 continue;
213             }
214             int burst_index = 0;
215             int burst_total = 0;
216             const bool burst = parse_burst_tag(frame.payload, burst_index,
                ↪  burst_total);
217             if (!burst || burst_index == 1 || burst_index == burst_total ||
                ↪  burst_index % 20 == 0) {
218                 log(L" 转发 " + utf8_to_wide(outgoing));
219             }
220             broadcast(client, MsgType::Chat, outgoing);
221         }
```

```
void ChatServer::broadcast(const std::shared_ptr<Client>& except, MsgType type,
    const std::string& payload) {
    std::vector<std::shared_ptr<Client>> snapshot;
    {
        std::lock_guard<std::mutex> lock(clients_mu_);
        snapshot = clients_;
    }
    for (const auto& client : snapshot) {
        if (client == except || !client->introduced) {
            continue;
        }
        send_to(client, type, payload);
    }
}
```

4.4 客户端的序号检查与退出

接收线程每得到一帧就调用 `note_seq`。期望序号从 1 开始，收到的序号等于期望值时只把期望值加 1；否则把缺口累加到 `gaps`，并把下一次期望改成当前序号的下一个。状态栏根据 `seq_ok` 显示“序号连续”或“序号空洞”。CHAT 负载里的 `burst i/n` 另外用一个标记数组统计测试条数，避免同一序号被重复计入。

```
void ChatClient::note_seq(uint32_t seq) {
    std::lock_guard<std::mutex> lock(stats_mu_);
    ++stats_.frames;
    if (seq != expected_seq_) {
        stats_.seq_ok = false;
        if (seq > expected_seq_) {
            stats_.gaps += seq - expected_seq_;
        } else {
            ++stats_.gaps;
        }
        expected_seq_ = seq + 1;
    } else {
        ++expected_seq_;
    }
}
```

断开时先在发送锁内发出 QUIT 并 `shutdown`，释放锁之后再 `join` 连续发送线程和接收线程，最后 `closesocket`。发送锁不能在 `join` 期间一直持有，否则接收线程若还要取这把锁就会死锁。

4.5 界面线程

套接字线程不接触控件。服务端的日志回调和在线名单回调、客户端的消息回调，都通过 `post` 把闭包排到 UI 线程。`open_` 在窗口关闭时置假，避免关闭过程中再改已经销毁的控件。

```
    void MainWindow::post(std::function<void()> action)
    {
```

```
167         if (!open_.load())
168         {
169             return;
170         }
171         DispatcherQueue().TryEnqueue([this, action = std::move(action)] {
172             if (!open_.load())
173             {
174                 return;
175             }
176             action();
177         });
178     }
```

5 界面与测试

服务端窗口有端口、开始监听、停止、在线用户和日志，见图 1。客户端窗口有地址、端口、昵称、消息列表、输入框和连续发送，状态栏在最下方。

三名用户同时连到 `127.0.0.1:9000`：张三、Alice 和李四。张三发送 `Hello` 和“你好，晚上见”，Alice 回复 `Good evening` 晚上好。Alice 的窗口同时出现英文和中文，见图 2。协议负载始终是 UTF-8 字节，WinUI 文本框本身使用 Unicode，中文不依赖控制台代码页。



图 1: 服务端正在监听 9000。在线用户为张三、Alice、李四，日志中有中文转发记录。

李四点击“在线列表”后，收到“在线用户：张三、Alice、李四”，见图 3。张三点“断开”之后，Alice 收到“张三离开了聊天”，见图 4。服务端在线名单随之去掉张三，并在日志中记下离开，见图 5。



图 2: Alice 收到张三的英文和中文，并发送了中英混合的回复。



图 3: 李四收到完整在线名单，状态栏显示序号连续，连续发送 20 条全部到达。



图 4: 张三断开后，Alice 收到离开通知。此前的英文和中文消息都还在。



图 5: 张三离开后，在线用户剩下 Alice 和李四。

6 序号连续性

序号属于某一条连接上的一个方向，从 1 开始，每成功组一帧加 1。接收方若发现新序号不是上一个序号加 1，就把该间隔计为序号空洞。

界面上的“连续发送”让 Alice 连续发出 20 条 `burst i/20`。张三的状态栏为：已发送 2，已收到帧 24（聊天 21），序号连续，连续发送收到 20/20，见图 6。20 条测试消息和它前面的聊天帧都按序号到齐。李四一侧同样是“序号连续”和“连续发送收到 20/20”。

另有一个不带界面的环回程序，在本机用同一套服务端和客户端核心发送 100 条。运行结果为：

```
英文消息: Hello
中文消息: 你好，晚上见 / Good evening 晚上好
在线用户包含张三、Alice、李四
未连接 | 已发送 1 | 已收到帧 105（聊天 102） | 序号连续 | 连续发送收到 100/100
序号空洞: 0
PASS
```



图 6: 张三收到 Alice 的连续发送。状态栏显示序号连续，20 条全部到达。

TCP 会重传丢失的报文段。连接一直活着时，应用层读到的字节流是完整的，所以帧序号保持连续。若去掉长度字段，同一次 `recv` 可能把两条消息粘在一起，或者只拿到一条消息的前半段，看起来像内容损坏。那是字节流的分段，序号空洞并不会因此出现。

连接被强行掐断时，还没进入对端接收缓冲的数据会随这条连接消失。接收方看到的是 `recv` 返回 0 或错误，然后连接结束。这和链路上随机丢掉一个报文段不同：后者由 TCP 重传补上，前者是连接本身被取消了。

7 结论

实验用固定长度帧把 TCP 字节流还原成一条条聊天消息，并用每个方向独立递增的序号检查有没有缺帧。英文、中文和三人同时在线都工作正常。界面连续发送 20 条、环回测试连续发送 100

条时，接收方的序号都没有空洞，收到的测试条数与发送条数一致。连接存活期间，应用层观察不到消息的丢失。